

Ocaml Project - Akleenator

Edgar Delaporte edgar.delaporte@enac.fr

November 2025

Summary

The purpose of this project is to implement the **L*** (L-star) **active learning** algorithm in **OCaml**. This particular type of machine learning makes it possible to 'guess' in finite time an infinite language known by a black box that can be queried. This allows an opaque system to be analysed by reconstructing an equivalent transparent system.

Context

A **black box** is a system whose internal logic is hidden or unknown. One can observe its inputs and outputs, but not how it transforms one into the other. Many real-world systems behave like black boxes, such as neural networks or proprietary APIs. Being able to extrapolate the behavior of a black box is therefore essential for analyzing them.

The **L*** **active learning** algorithm, introduced by Dana Angluin in 1987, is capable of learning the behavior of a black box if the latter behaves like a Deterministic Finite Automaton (**DFA**).

i.e. if a black box recognises a regular language (infinite set of words), **L*** is capable of producing the equivalent minimal DFA in finite time. In the context of **L***, we call the black box the **oracle**.

The Algorithm

L* works by asking membership queries to the oracle. This consists in presenting the oracle with a word and having it tell whether or not it recognises it.

L* uses these answers to build an **observation table** from which it can infer a DFA hypothesis.

It can then compare its hypothesis to the oracle through various methods. If this hypothesis is valid then it returns it and terminates, else it is able to find a **counter-example** and use it to refine the observation table further.

The original pseudo-code for this algorithm and example runs are available [here](#). As the algorithm's operation is not trivial, do not hesitate to ask the project manager to explain

it to you in detail before you start reading any pseudo-code.

Requirements

You must write a program capable of learning the language of an oracle using the **L*** algorithm. The program must produce logs of all queries made and finish by graphically displaying the resulting DFA.

The logic of the oracle will be provided to you so that you only have to focus on the learning part.

A naive implementation of the algorithm probably causes exponential execution time. An optimal implementation, on the other hand, allows for execution time bounded by a polynomial function of the number of states of the oracle and letters in the alphabet. This requires certain optimisations, some of which are simple and others much less so. A perfect and rapid implementation of the algorithm is not required, but efforts and considerations towards optimisation are expected.